

Week 7: Running the Apps in VS Code

Dr M AOUE

Lecture notes

Goal. Run the Week 7 examples in VS Code and compare their behavior with the lecture. Start with the package apps; configure Google Maps separately. The code is already supplied, so you do not need to recreate the area package.

1 Open the package-practice project

Open code/week07/examples as a folder in VS Code. Open its integrated terminal; confirm that pubspec.yaml is in the current directory. Install dependencies and run the checks:

```
1 flutter pub get
2 flutter analyze
3 flutter test
```

To add a web runner for the first time:

```
1 flutter create --platforms=web .
2 flutter run -d chrome
```

Keep the supplied source and tests. The default entry point starts the area calculator. Use `flutter devices` to list available targets and the VS Code device selector to choose one.

2 Run the package examples

2.1 Number formatting

```
1 flutter run -t lib/00_number_format.dart -d chrome
```

Expect **US: 1,234.57**, **German: 1.234,57**, and **Area: 12.3457**. The first two format the same number using different locale conventions.

2.2 HTTP request

```
1 flutter run -t lib/01_http_request.dart -d chrome
```

Expect a loading indicator, then a title from the demonstration service. If the request fails, the app displays an error and Retry. Internet and the server's availability affect this example. Browser requests must satisfy CORS. See the project README for Android release Internet permission and macOS networking configuration.

2.3 Area calculator

```
1 flutter run -t lib/02_area_calculator.dart -d chrome
```

Enter width 4 and height 3. Rectangle gives 12 square units; Triangle gives 6. Try a blank field, zero, a negative number and NaN: the app reports invalid dimensions. Decimals use a point in this input exercise.

3 Inspect and check the area package

In VS Code, open `packages/area/lib/area.dart`. Follow its exports to the calculations and formatting functions. The parent pubspec refers to this package by a local path.

```
1 cd packages/area
2 dart pub get
3 dart analyze
4 dart test
```

These commands run in the Dart package directory, not the Flutter application's root. There is no device selector for its unit tests. The package remains private through `publish_to: 'none'`; no publishing command is required.

4 Prepare the maps project

Open `code/week07/maps_examples` as a separate VS Code folder. Run:

```
1 flutter pub get
2 flutter create --platforms=android,web .
3 flutter analyze
4 flutter test
```

On a Mac, generate an iOS runner if needed. The map plugin targets Android, iOS and web; do not select Windows desktop for these examples.

The map apps need your configured Maps API key and a suitable Google Cloud billing setup. No key or cloud project is included. Follow the detailed platform instructions in the maps project's README before running the map.

For Android, enable Maps SDK for Android and add the restricted key metadata inside the manifest's application element. For iOS, enable Maps SDK for iOS and supply its restricted key in the application launch configuration. For web, enable Maps JavaScript API and load its script before the Flutter bootstrap. Use separate platform restrictions appropriate to each key. The current official setup is linked from the README.

The location exercise additionally requires foreground-location permissions and a meaningful iOS usage description. Browser geolocation needs HTTPS or localhost and permission from the user. The sample never requests background tracking.

5 Run the map examples in order

5.1 First verify the fixed map

```
1 flutter run -t lib/00_fixed_map.dart -d chrome
```

Expect a map centered on Beirut; pan and zoom it. No location prompt is required. A blank map can indicate a missing or restricted key, disabled API, billing problem, network failure or unsupported platform. Read the browser console or device logs to determine which condition applies.

5.2 Then request the device location

```
1 flutter run -t lib/01_user_location.dart -d chrome
```

Tap **Find my location**. On success the camera moves and the app displays coordinates. On denial or disabled services it shows a visible error. Change browser/device settings before retrying if permission is permanently denied. Beirut remains a labeled starting point, not a claim about your actual location.

5.3 Add sample markers

```
1 flutter run -t lib/02_markers.dart -d chrome
2 flutter run -d chrome
```

The first command runs the sample-marker example; the second runs the combined app. Stop one run before starting the other. Tap the places action to toggle two invented pins near Beirut and tap a pin for its info window. If the combined app moves to a distant user location, pan back to Beirut to see those sample pins. These are teaching data, not live Places search results.

6 What the checks verify

The package-app tests simulate the calculator and check formatting and HTTP retry using a fake response. The area-package tests check numeric results and invalid dimensions. Location tests use a fake gateway to check permission and service branches.

Passing those checks does not validate the Maps key, native view, billing configuration or actual GPS. Finish by manually running the map on your configured target. Live nearby restaurant search is an optional backend extension explained in the lecture; no backend is supplied.

7 Common setup problems

- **No pubspec found:** change to the correct project directory.
- **Package import unresolved:** check YAML indentation and run `flutter pub get`.
- **No platform runner:** generate the intended runner before launching.
- **Plugin missing after an edit:** stop the app and rebuild; hot reload does not register new native plugins.
- **Location unavailable:** check device services, foreground permission and browser secure-context requirements.
- **Dependency resolution failure:** check the SDK requirement and conflicting constraints; do not remove the lockfile as the first response.